

Lekcja 12: Zaawansowane OOP i Obsługa Wyjątków

#lekcja #python #oop #wyjątki #klasy #dataclass

W tej lekcji zagłębimy się w bardziej zaawansowane aspekty programowania obiektowego w Pythonie. Poznamy różnice między metodami statycznymi a metodami klas, nauczymy się tworzyć wygodne klasy danych oraz dowiemy się, jak profesjonalnie zarządzać błędami za pomocą wyjątków.

1. Metody statyczne, metody klasy i property

W klasach, oprócz standardowych metod instancji (które operują na `self`), możemy definiować specjalne rodzaje metod.

Metody statyczne (`@staticmethod`)

Definition

Metoda statyczna to funkcja zdefiniowana wewnątrz klasy, która nie ma dostępu do instancji (`self`) ani do samej klasy (`cls`). Działa jak zwykła funkcja, ale jest logicznie powiązana z klasą i umieszczona w jej przestrzeni nazw.

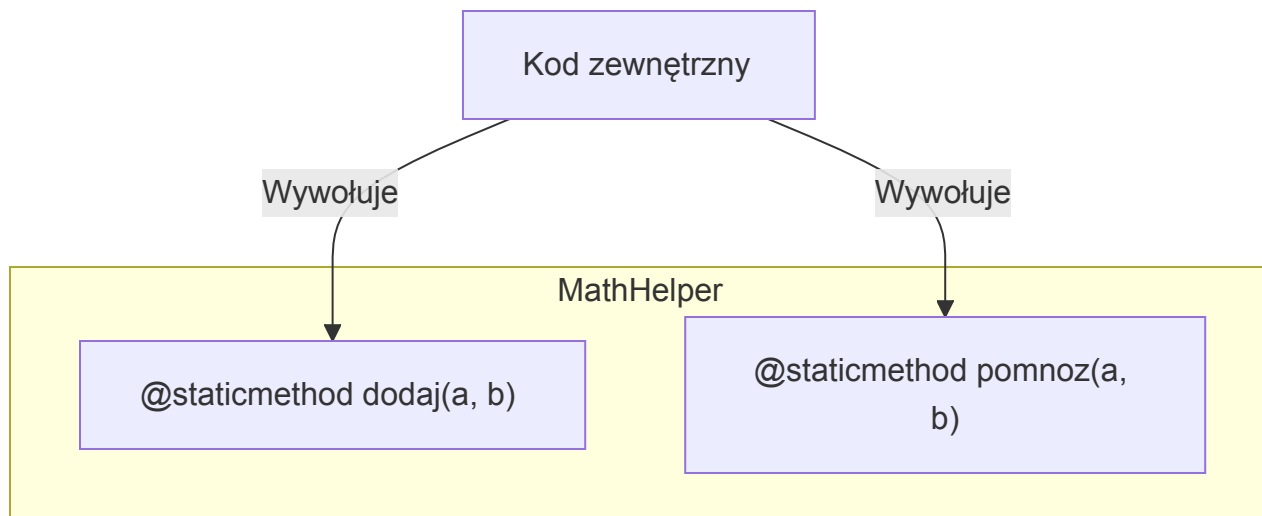
Używamy ich do tworzenia funkcji pomocniczych, które mają sens w kontekście klasy, ale nie potrzebują danych z konkretnego obiektu.

```
# Przykład: Klasa z narzędziami matematycznymi
class MathHelper:
    @staticmethod
    def dodaj(a, b):
        """Ta metoda nie potrzebuje 'self'. Działa jak zwykła funkcja."""
        return a + b

    @staticmethod
    def pomnoz(a, b):
        """Możemy ją wywołać bezpośrednio na klasie."""
        return a * b

# Wywołanie metod statycznych bez tworzenia obiektu
suma = MathHelper.dodaj(5, 10)
iloczyn = MathHelper.pomnoz(3, 7)
```

```
print(f"Suma: {suma}") # -> Suma: 15
print(f"Iloczyn: {iloczyn}") # -> Iloczyn: 21
```



Metody klasy (@classmethod)

Definition

Metoda klasy otrzymuje jako pierwszy argument samą klasę (cls), a nie instancję (self). Jest używana, gdy metoda musi operować na klasie, na przykład do tworzenia obiektów w niestandardowy sposób (alternatywne konstruktory).

```
# Przykład: Tworzenie obiektu z różnych formatów danych
```

```
import datetime
```

```
class Osoba:
```

```
    def __init__(self, imie, wiek):
        self.imie = imie
        self.wiek = wiek
```

```
    @classmethod
```

```
    def z_roku_urodzenia(cls, imie, rok_urodzenia):
        """
```

```
        Alternatywny konstruktor.
        'cls' to referencja do klasy Osoba.
        """
```

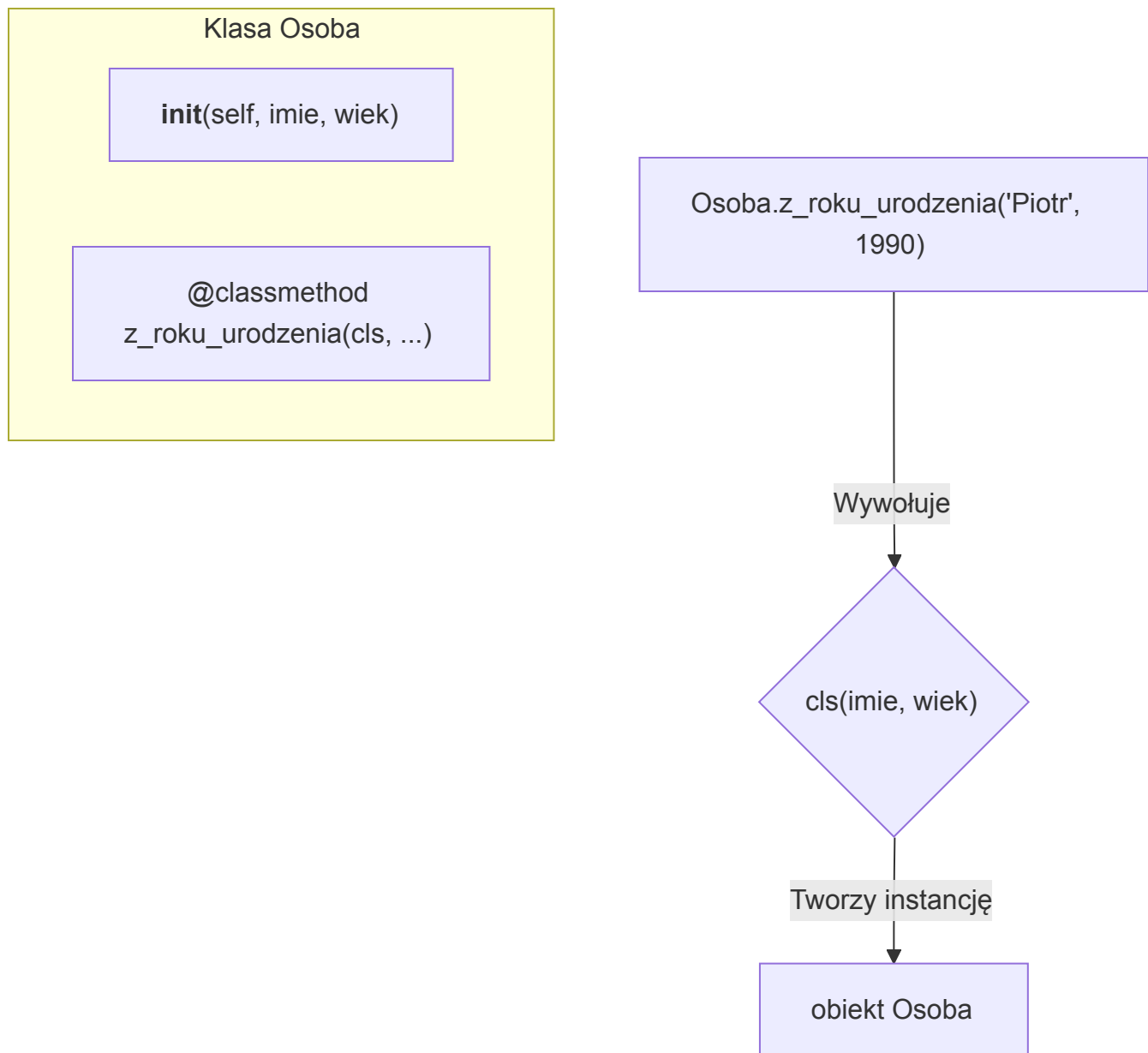
```
        aktualny_rok = datetime.date.today().year
        wiek = aktualny_rok - rok_urodzenia
        # Zwracamy nową instancję klasy
        return cls(imie, wiek)
```

```
# Standardowe tworzenie obiektu
```

```
osoba1 = Osoba("Anna", 30)

# Tworzenie obiektu za pomocą metody klasy
osoba2 = Osoba.z_roku_urodzenia("Piotr", 1990)

print(f"{osoba1.imie} ma {osoba1.wiek} lat.") # -> Anna ma 30 lat.
print(f"{osoba2.imie} ma {osoba2.wiek} lat.") # -> Piotr ma 34 lat.
```



Właściwości (`@property`)

 Definition

@property to dekorator, który pozwala traktować metodę klasy jak atrybut. Umożliwia kontrolę dostępu do atrybutów instancji, np. poprzez dodanie logiki (walidacji) podczas odczytu, zapisu lub usuwania wartości.

Jest to bardziej "pythonowy" sposób na implementację getterów i setterów.

```
# Przykład: Kontrola dostępu do atrybutu
class Pracownik:
    def __init__(self, imie, pensja):
        self.imie = imie
        self._pensja = pensja # Używamy "_" by zasugerować, że to atrybut
                              "prywatny"

    @property
    def pensja(self):
        """Getter – wywoływany przy odczycie atrybutu."""
        print("Odczytano pensję...")
        return self._pensja

    @pensja.setter
    def pensja(self, nowa_wartosc):
        """Setter – wywoływany przy próbie przypisania nowej wartości."""
        if nowa_wartosc < 0:
            print("Błąd: Pensja nie może być ujemna!")
        else:
            print("Ustawiono nową pensję...")
            self._pensja = nowa_wartosc

jan = Pracownik("Jan", 5000)

# Odczyt wartości (wywołuje getter)
print(f"Pensja Jana: {jan.pensja}") # -> Odczytano pensję... -> Pensja Jana:
5000

# Przypisanie nowej wartości (wywołuje setter)
jan.pensja = 6000 # -> Ustawiono nową pensję...

# Próba przypisania nieprawidłowej wartości
jan.pensja = -100 # -> Błąd: Pensja nie może być ujemna!

print(f"Aktualna pensja: {jan.pensja}") # -> Odczytano pensję... -> Aktualna
pensja: 6000
```

2. Klasy danych (@dataclass)

Definition

Klasa danych to zwykła klasa, której głównym celem jest przechowywanie danych (stanu). Dekorator `@dataclass` z modułu `dataclasses` automatycznie generuje za nas standardowe metody, takie jak `init()`, `repr()`, `eq()` i inne.

Dzięki temu kod jest znacznie krótszy i bardziej czytelny.

```
from dataclasses import dataclass

# Tradycyjne podejście
class ProduktTradycyjny:
    def __init__(self, nazwa, cena, ilosc):
        self.nazwa = nazwa
        self.cena = cena
        self.ilosc = ilosc

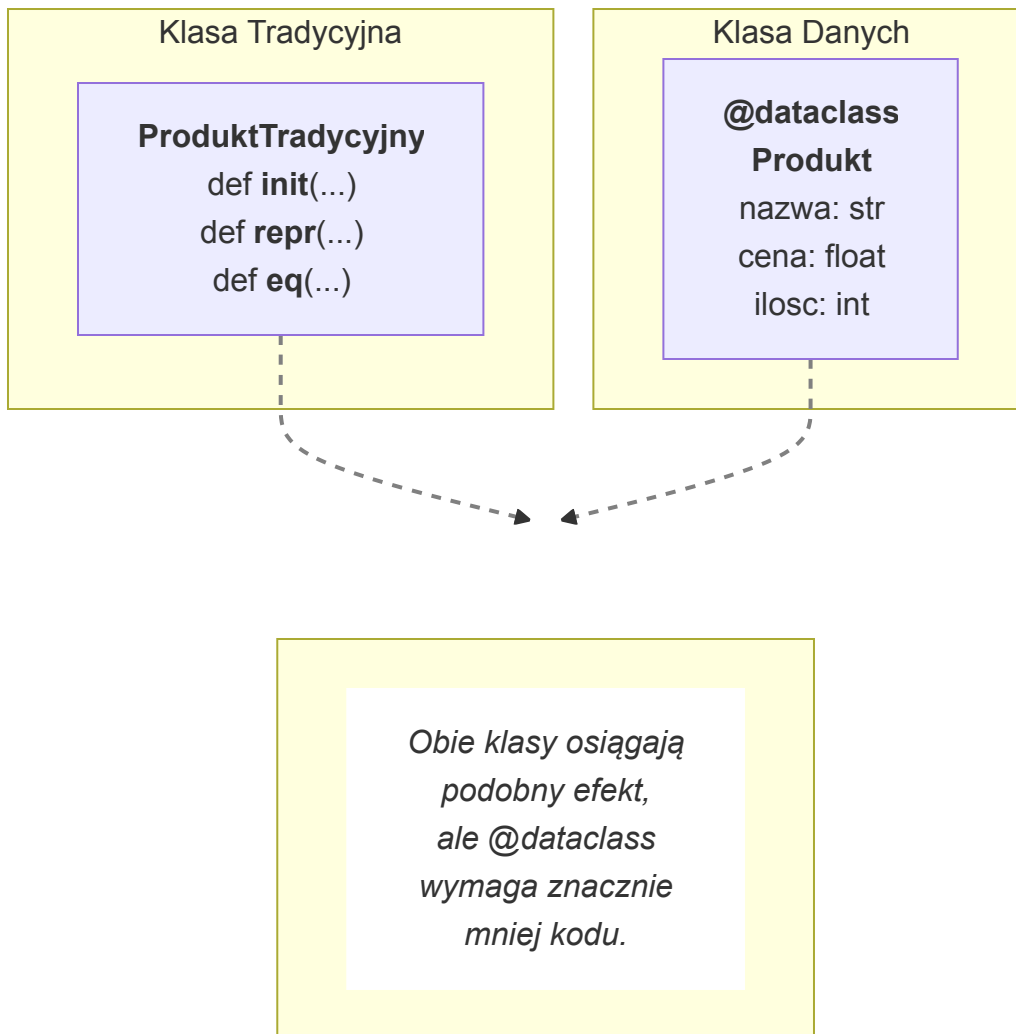
    def __repr__(self):
        return f"ProduktTradycyjny(nazwa='{self.nazwa}', cena={self.cena}, ilosc={self.ilosc})"

    def __eq__(self, other):
        return (self.nazwa, self.cena, self.ilosc) == (other.nazwa, other.cena, other.ilosc)

# Podejście z @dataclass
@dataclass
class Produkt:
    nazwa: str
    cena: float
    ilosc: int = 0 # Można zdefiniować wartości domyślne

# @dataclass automatycznie stworzył __init__, __repr__, __eq__ i inne!
jablko1 = Produkt("Jabłko", 2.50, 10)
jablko2 = Produkt("Jabłko", 2.50, 10)
banan = Produkt("Banan", 4.99)

print(jablko1) # -> Produkt(nazwa='Jabłko', cena=2.5, ilosc=10)
print(banan)   # -> Produkt(nazwa='Banan', cena=4.99, ilosc=0)
print(jablko1 == jablko2) # -> True
```



3. Metaklasy

Info

Metaklasy to zaawansowany koncept w Pythonie. W 99% przypadków nie będziesz musiał ich używać, ale warto wiedzieć, że istnieją.

Definition

Metaklasa to "klasa, która tworzy klasy". Tak jak klasa jest szablonem do tworzenia obiektów, tak metaklasa jest szablonem do tworzenia klas. Pozwalają one na modyfikowanie procesu tworzenia klasy w momencie jej definiowania.

Domyślną metaklasą w Pythonie jest `type`. Kiedy definiujesz klasę, Python w tle używa `type` do jej stworzenia.

```

# Przykład koncepcyjny: Prosta metaklasa, która dodaje atrybut do każdej
# tworzonej klasy

# Definiujemy metaklasę, która dziedziczy po 'type'
class MetaDodajAtrybut(type):
    def __new__(cls, name, bases, dct):
        # dct to słownik atrybutów i metod klasy
        dct['autor'] = 'Kurs Python' # Dodajemy nowy atrybut
        # Wywołujemy __new__ z nadrzędnej metaklas (type)
        return super().__new__(cls, name, bases, dct)

# Używamy naszej metaklas do stworzenia nowej klasy
class MojaKlasa(metaclass=MetaDodajAtrybut):
    pass

class InnaKlasa(metaclass=MetaDodajAtrybut):
    opis = "To jest inna klasa."

# Każda klasa stworzona z tą metaklasą będzie miała atrybut 'autor'
print(MojaKlasa.autor) # -> Kurs Python
print(InnaKlasa.autor) # -> Kurs Python
print(InnaKlasa.opis) # -> To jest inna klasa.

```

4. Obsługa wyjątków: `try...except`

Definition

Wyjątki to błędy wykryte podczas wykonywania programu. Kiedy wystąpi błąd, Python "podnosi" (ang. raises) wyjątek. Jeśli nie zostanie on "obsłużony", program zakończy swoje działanie. Blok `try...except` służy do przechwytywania i obsługi tych wyjątków.

Struktura bloku:

- **try** : Umieszczamy tu kod, który może potencjalnie spowodować błąd.
- **except** : Ten blok jest wykonywany tylko wtedy, gdy w bloku **try** wystąpił określony wyjątek.

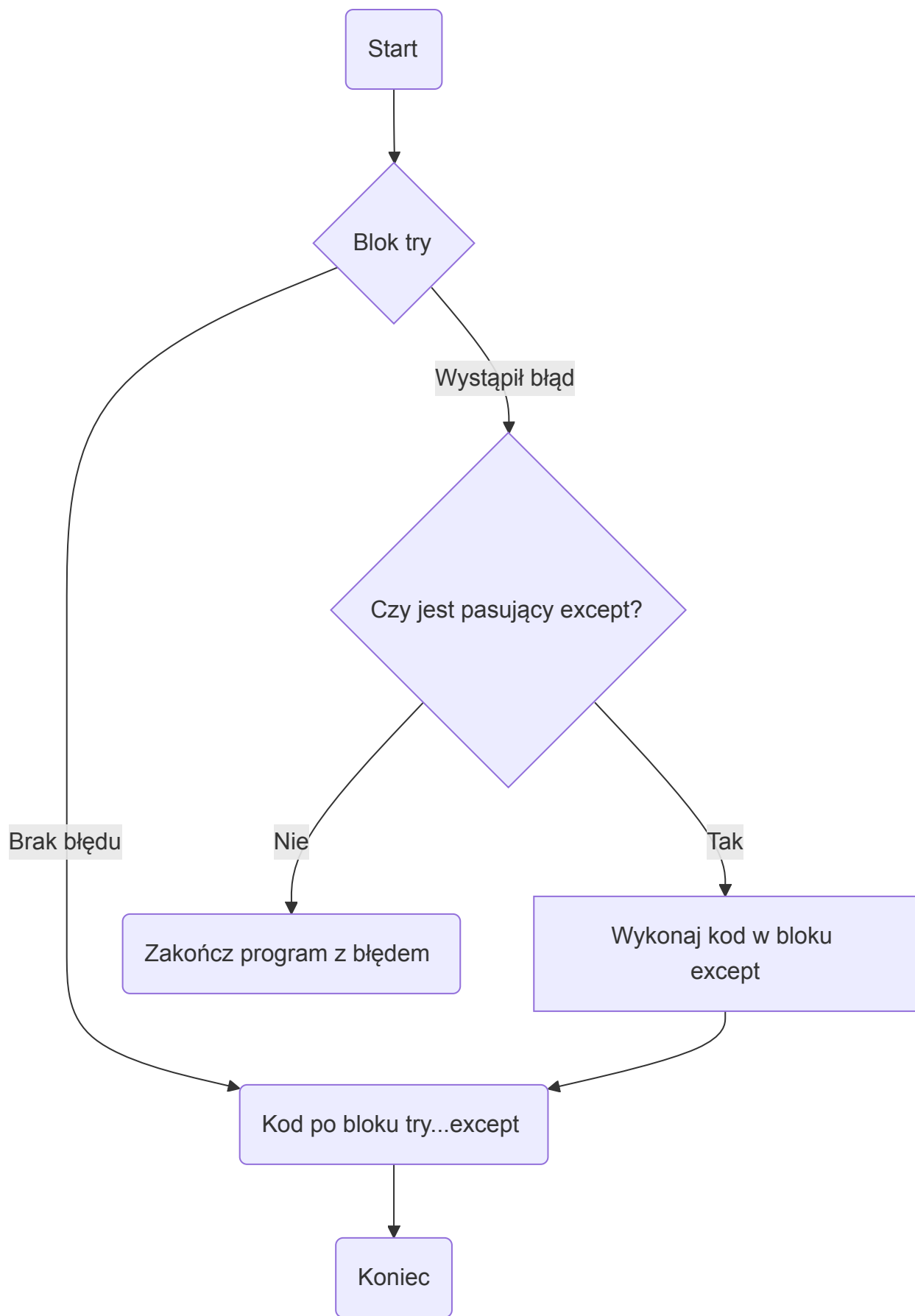
```

# Przykład: Dzielenie przez zero
try:
    liczba = 10
    wynik = liczba / 0 # Ta linia spowoduje błąd ZeroDivisionError
    print("Ten tekst się nie wyświetli.")
except ZeroDivisionError:

```

```
# Ten blok zostanie wykonany, bo przechwyciliśmy właściwy błąd
print("Błąd: Nie można dzielić przez zero!")

print("Program działa dalej.")
```



Możemy obsługiwać wiele różnych wyjątków:

```

try:
    lista = [1, 2, 3]
    wartosc = int(input("Podaj liczbę lub indeks: "))
    print(lista[wartosc]) # Może rzucić IndexError
    print(10 / wartosc)   # Może rzucić ZeroDivisionError
except ValueError:
    print("To nie była liczba!")
except IndexError:
    print("Nie ma takiego indeksu w liście!")
except ZeroDivisionError:
    print("Nie można dzielić przez zero!")
except Exception as e: # Przechwytuje wszystkie inne błędy
    print(f"Wystąpił nieoczekiwany błąd: {e}")

```

Bloki else i finally

Tip

Bloki else i finally dają dodatkową kontrolę nad przepływem sterowania w obsłudze wyjątków.

- **else** : Wykonywany, jeśli blok **try** zakończył się **bez** żadnego błędu.
- **finally** : Wykonywany **zawsze**, niezależnie od tego, czy wystąpił błąd, czy nie. Idealne miejsce na "sprzątanie", np. zamykanie plików.

```

plik = None # Zmienna zdefiniowana na zewnątrz, by była widoczna w finally
try:
    plik = open("dane.txt", "r", encoding="utf-8")
    zawartosc = plik.read()
except FileNotFoundError:
    print("Błąd: Plik nie istnieje.")
else:
    # Ten blok wykona się tylko, jeśli plik został pomyślnie otwarty
    print("Plik odczytany pomyślnie.")
    print("Zawartość:", zawartosc)
finally:
    # Ten blok wykona się zawsze
    if plik:
        plik.close()
        print("Blok finally: Plik został zamknięty.")
    else:
        print("Blok finally: Nie było czego zamykać.")

```

Tworzenie własnych wyjątków

Możemy tworzyć własne, niestandardowe klasy wyjątków, dziedzicząc po wbudowanej klasie `Exception`. Pozwala to na tworzenie bardziej czytelnego i specyficznego kodu.

```
# 1. Tworzymy własną klasę wyjątku
class WartoscUjemnaError(Exception):
    """Wyjątek podnoszony, gdy wartość jest niedozwolona (ujemna)."""
    pass # Nie potrzebujemy dodatkowej logiki, wystarczy sama nazwa

# 2. Funkcja, która może podnieść nasz wyjątek
def oblicz_pierwiastek(liczba):
    if liczba < 0:
        # Używamy słowa kluczowego 'raise' do podniesienia wyjątku
        raise WartoscUjemnaError(f"Nie można obliczyć pierwiastka z liczby ujemnej: {liczba}")
    return liczba ** 0.5

# 3. Obsługa naszego wyjątku
try:
    print(oblicz_pierwiastek(25))
    print(oblicz_pierwiastek(-4))
except WartoscUjemnaError as e:
    print(f"Wystąpił błąd aplikacji: {e}")
```

Zadania do samodzielnej pracy

Zadania proste

1. Zadanie 1 – Klasa danych Film

Stwórz klasę danych (`@dataclass`) o nazwie `Film`, która będzie przechowywać tytuł (`string`), reżysera (`string`) i rok produkcji (`integer`). Utwórz dwie instancje tej klasy i wyświetl je.

2. Zadanie 2 – Walidator wieku

Stwórz klasę `Uzytkownik` z atrybutem `_wiek`. Użyj dekoratora `@property`, aby stworzyć właściwość `wiek`. Getter powinien zwracać wiek, a setter powinien sprawdzać, czy podany wiek jest w zakresie od 0 do 120. Jeśli nie jest, powinien wyświetlić komunikat błędu i nie zmieniać wartości.

3. Zadanie 3 – Konwerter Walut

Stwórz klasę `KalkulatorWalut`. Dodaj w niej metodę statyczną (`@staticmethod`) o nazwie `usd_na_pln`, która przyjmuje kwotę w dolarach i zwraca ją przeliczoną na złotówki (przyjmij stały kurs, np. 1 USD = 4.0 PLN). Wywołaj tę metodę bez tworzenia obiektu klasy.

4. 🖋️ Zadanie 4 – Bezpieczne dzielenie

Napisz funkcję `bezpieczne_dzielenie(a, b)`, która zwraca wynik dzielenia a / b . Użyj bloku `try...except`, aby obsłużyć błąd `ZeroDivisionError`. Jeśli wystąpi ten błąd, funkcja powinna zwrócić `None` i wyświetlić komunikat "Błąd: Dzielenie przez zero!".

5. 🖋️ Zadanie 5 – Odczyt pliku

Napisz program, który próbuje otworzyć i odczytać plik o nazwie `nieistniejacy.txt`. Użyj bloku `try...except`, aby obsłużyć wyjątek `FileNotFoundError` i wyświetlić przyjazny komunikat użytkownikowi.

Zadania-wyzwania (challenge)

6. 🧠 Zadanie 6 – Własny wyjątek `InvalidPasswordError`

Stwórz własny wyjątek `InvalidPasswordError`. Następnie napisz funkcję `ustaw_haslo(haslo)`, która sprawdza, czy hasło ma co najmniej 8 znaków. Jeśli nie, funkcja powinna podnieść (raise) wyjątek `InvalidPasswordError` z odpowiednim komunikatem. Napisz kod, który testuje tę funkcję w bloku `try...except`.

7. 🧠 Zadanie 7 – Alternatywny konstruktor dla `Data`

Stwórz klasę `Data` z atrybutami `dzien`, `miesiac`, `rok`. Dodaj metodę klasy (`@classmethod`) o nazwie `ze_stringa`, która przyjmuje datę w formacie "DD-MM-RRRR" (np. "25-12-2023") i tworzy na jej podstawie obiekt klasy `Data`. Pamiętaj o konwersji typów na `int`.

8. 🧠 Zadanie 8 – Kalkulator z pełną obsługą błędów

Stwórz prosty kalkulator, który prosi użytkownika o podanie dwóch liczb i operacji (+, -, *, /).

- Całość umieść w pętli `while True`, aby program działał do momentu przerwania.
- Użyj bloku `try...except` do obsługi:
 - `ValueError`, jeśli użytkownik wpisze coś, co nie jest liczbą.
 - `ZeroDivisionError` przy próbie dzielenia przez zero.
- Użyj bloku `else`, aby wyświetlić wynik tylko wtedy, gdy nie było błędu.
- Użyj bloku `finally`, aby na koniec każdej iteracji pętli wyświetlić komunikat "Koniec obliczeń."

9. 🧠 Zadanie 9 – Klasa `KontoBankowe` z property i wyjątkami

Stwórz klasę `KontoBankowe` za pomocą `@dataclass`, która ma atrybut `_saldo` (prywatne).

- Stwórz właściwość (`@property`) `saldo`, która tylko odczytuje wartość `_saldo`.
- Stwórz metodę `wplac(kwota)`, która dodaje kwotę do salda. Metoda powinna podnosić `ValueError`, jeśli kwota jest ujemna.
- Stwórz metodę `wypłac(kwota)`, która odejmuje kwotę od salda. Metoda powinna podnosić `ValueError`, jeśli kwota do wypłaty jest ujemna, oraz własny wyjątek

`BrakSrodkowError` , jeśli saldo jest niewystarczające.

- Przetestuj działanie klasy, obsługując wszystkie możliwe wyjątki.

10. 🧠 Zadanie 10 – Metaklasa walidująca

Stwórz metaklasę `MetaWalidujMetody`, która podczas tworzenia nowej klasy sprawdza, czy wszystkie jej metody (poza metodami "magicznymi", czyli zaczynającymi się od `__`) mają docstring. Jeśli któraś metoda go nie ma, metaklasa powinna podnieść `TypeError` z informacją, która metoda wymaga dokumentacji. Przetestuj ją, tworząc klasę z poprawnie i niepoprawnie udokumentowanymi metodami.